LABORATORY OF AUTOMATION SYSTEMS

Samuele Pilati

Giorgio Giaretta

Tommaso Magelli



Samuele Pilati
Giorgio Giaretta
Tommaso Magelli

INDEX

1. INTRODUCTION.....
2. MOTOR MODEL.....
3. INTERFACING BLOCK BETWEEN MICROCONTROLLER AND MOTOR.....
4. Pulse Width Modulation (PWM) .....
5. ENCODER.....
6. CASCADE CONTROL.....
7. FILTER.....
8. IMPLEMENTATION OF THE ANTI-WINDUP SCHEME.....
9. TRAJECTORY TRACKING AND FEEDFORWARD CONTROL.....
10. SYSTEM IDENTIFICATION VIA LEAST SQUARES ESTIMATION.....
11. ANALYTICAL DESIGN WITH A DEADBEAT CONTROLLER.....
12. ANALYTICAL DESIGN WITH A DAHLIN CONTROLLER.....
13. (BONUS) ONLINE PARAMETERS ESTIMATION.....
14. CONCLUSION.....

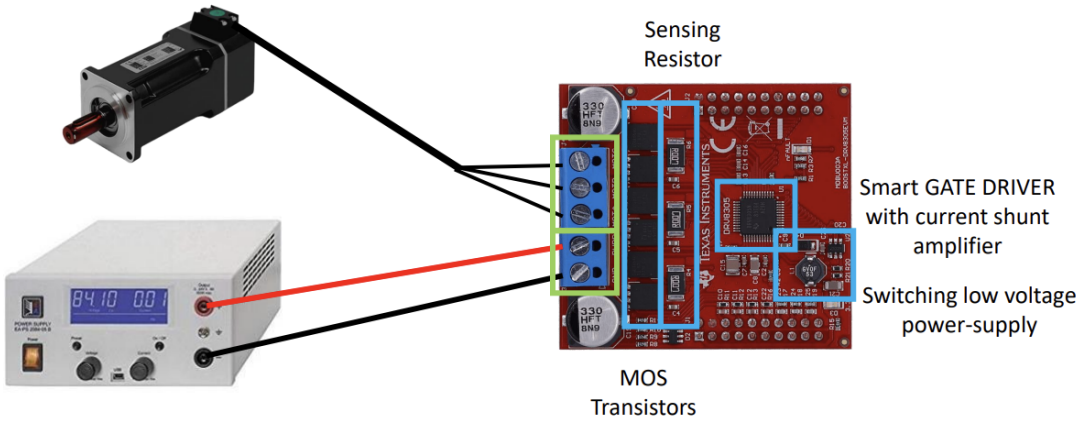
Samuele Pilati
Giorgio Giaretta
Tommaso Magelli

INTRODUCTION

1. Experimental Setup

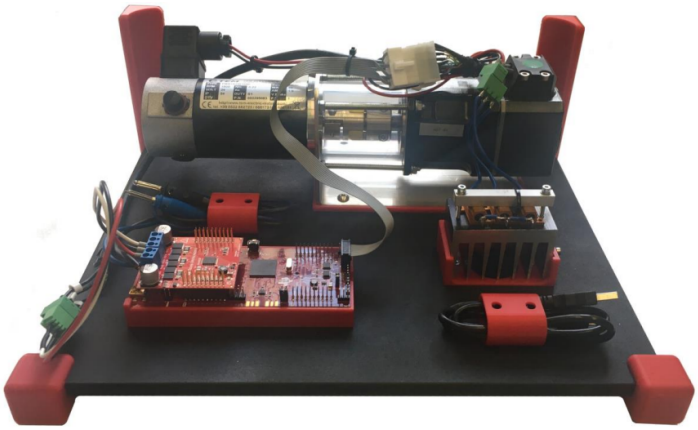
The experiments described in this report were conducted on a motor control setup designed to provide practical experience with real-world automation systems. The primary hardware components include:

- A **LAUNCHXL-F28379D** board, which served as the real-time microcontroller unit (MCU) where the control algorithms are deployed.
- A **BOOSTXL-DRV8305EVM** power driver, which acts as the power stage, converting the logic-level signals from the MCU into the high-power signals required to drive the motor.



- A **PMB 1524 DC Brushed Motor**.
- An incremental **quadrature encoder** attached to the motor for position and speed feedback.
- A **Brushless motor** coupled to the DC motor.
- A set of **Braking Resistors** to dissipate energy and simulate a mechanical load on the system.

Samuele Pilati
Giorgio Giaretta
Tommaso Magelli

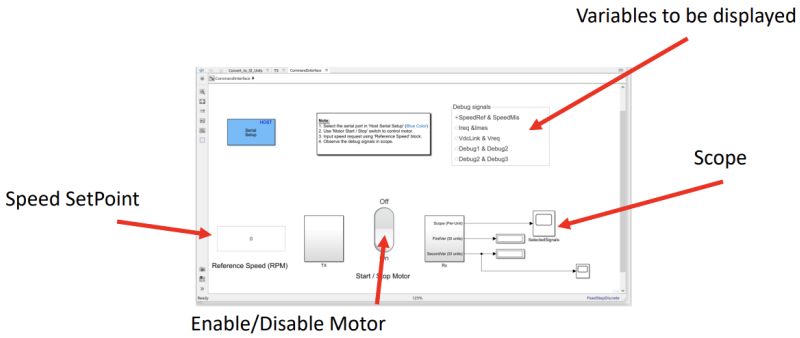


2. Development Environment

All control systems were designed and implemented using the **MATLAB R2023a** and **Simulink** environment. The C2000 Microcontroller Blockset was utilized to enable automatic C-code generation directly from the Simulink models. The generated code was then compiled and deployed to the LAUNCHXL-F28379D target.

The workflow involved two modes of operation:

1. **Monitor & Tune Mode:** Used for initial debugging and to get a low-refresh-rate overview of the system's behavior.
2. **Build deploy and start:** For high-performance analysis, the control algorithm was deployed to run independently on the MCU. A separate Simulink interface on the host PC communicated via a serial port to stream and log high-frequency data for detailed offline analysis in MATLAB.



3. Overview of Laboratory Activities

First, a mathematical model of the DC motor was developed in Simulink, using parameters derived from the manufacturer's datasheet. This model served as a baseline for understanding the system's theoretical behavior and for a virtual environment for initial testing.

Subsequently, we focused on the practical aspects of interfacing with the physical motor. This involved understanding the Pulse Width Modulation (PWM) technique used to drive the motor. We also analyzed the output from the encoder, implementing a low-pass filter to condition the noisy raw speed signal and make it suitable for feedback control.

We designed and implemented a standard cascaded PI control system. This architecture, featuring an inner current loop and an outer speed loop, was tuned using the pole-zero cancellation method. During this phase, we encountered the practical issue of actuator saturation and implemented a simple but effective anti-windup scheme to ensure stable operation.

To improve the system's performance, a feedforward action was designed. By using the reference trajectory's acceleration profile to pre-calculate the required motor torque, we were able to dramatically reduce the tracking error.

We then performed an experimental system identification using the Least Squares (LS) estimation algorithm. This allowed us to obtain a more accurate set of parameters (J, b, R, L) for our specific motor, leading to a more precise controller tuning.

Finally, we explored advanced analytical control techniques. A Deadbeat controller was implemented to achieve the fastest possible response. A Dahlin controller was also designed, which provided a smoother, tunable response by setting the closed loop to behave like a first-order system.

MOTOR MODEL

The initial phase of the laboratory work involved developing a mathematical model of the DC motor to serve as a baseline for simulation and controller design. The model is described by two interconnected dynamic systems: an electrical subsystem representing the armature circuit, and a mechanical subsystem representing the rotor dynamics.

The electrical dynamics are governed by Kirchhoff's voltage law applied to the armature circuit:

$$V_A(s) = R_A I_A(s) + sL_A I_A(s) + V_M(s)$$

where V_A is the applied armature voltage, I_A is the armature current, R_A and L_A are the armature resistance and inductance respectively, and V_M is the back-electromotive force (BEMF).

The mechanical dynamics are described by the rotational equation of motion:

$$sJ\Omega_m(s) = C_m(s) - b\Omega_m(s)$$

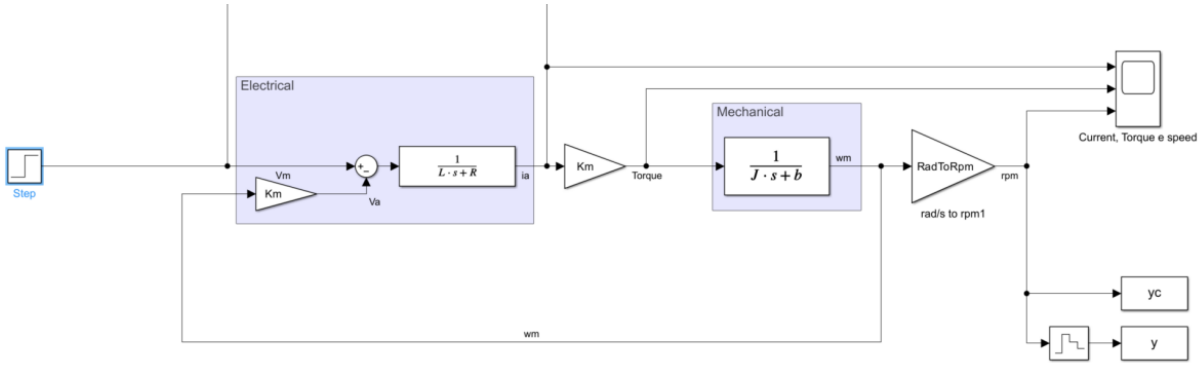
where J is the rotor inertia, b is the viscous friction coefficient, Ω_m is the angular velocity, and C_m is the motor torque.

These two domains are linked by two fundamental coupling equations, both dependent on the motor constant k_m:

1. The motor torque is proportional to the armature current: $C_m(s) = k_m I_A(s)$.
2. The back-EMF is proportional to the angular velocity: $V_M(s) = k_m \Omega_m(s)$.

These equations were implemented in a Simulink block diagram, creating a virtual representation of the motor that was used to test and validate control strategies before their deployment on the physical hardware.

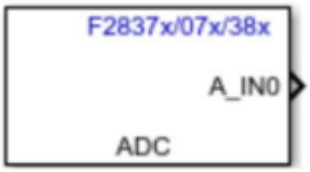
$$I_A(s) = \frac{V_A(s) - V_M(s)}{R_A + sL_A} \qquad \frac{\Omega_m(s)}{C_m(s)} = \frac{1}{b + sJ}$$



INTERFACING BLOCK BETWEEN MICROCONTROLLER AND MOTOR

Digital Output (GPIO): The GPIO DO block provides simple ON/OFF control over specific pins of the microcontroller. This is used for auxiliary functions, such as activating status LEDs or, more critically, sending the enable signal to the H-bridge motor driver, thus allowing power to flow to the motor.





Analog-to-Digital Converters (ADCs): For feedback control, the microcontroller needs to measure physical quantities from the motor, such as current and voltage. Since these are analog signals,

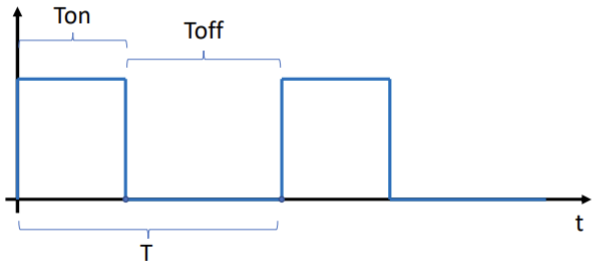
the ADC peripheral is used to convert them into digital numbers that the control algorithm can process.

(PWM): To control the motor's speed and torque, we must regulate the average voltage applied to its terminals. The ePWM (enhanced PWM) module does that by generating a high-frequency square wave. By modulating the duty cycle of this signal, the effective voltage delivered to the motor can be precisely controlled.



Pulse Width Modulation (PWM)

The primary method for controlling the power delivered to the DC motor in our setup is Pulse Width Modulation (PWM). This technique involves applying a high-frequency square wave voltage to the motor terminals instead of a continuous analog voltage. The average voltage, and thus the power delivered to the motor, is controlled by modulating the duty cycle of this signal, which is the percentage of time the signal remains in its high state within a single period.



$$period = Ton + Toff = T$$

$$frequency = \frac{1}{T}$$

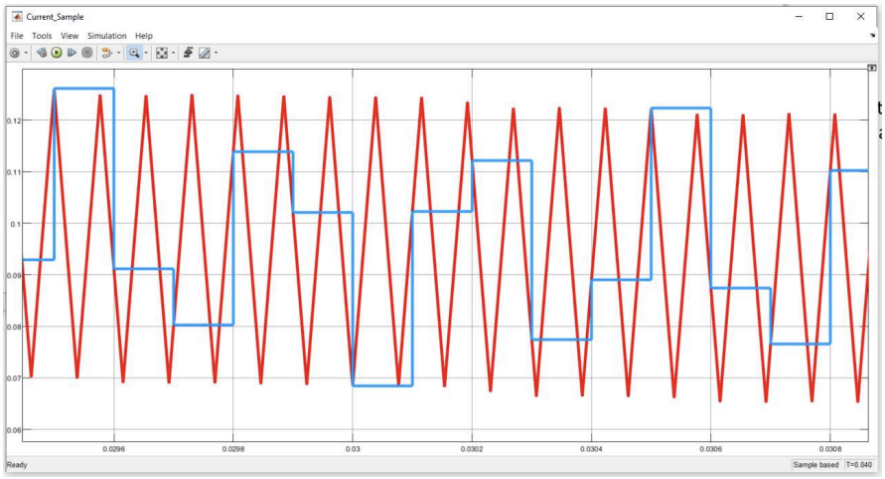
$$dutycycle = \frac{Ton}{Ton + Toff} * 100 = \frac{Ton}{T} * 100$$

A higher duty cycle corresponds to a higher average voltage, resulting in higher speed and torque, while a lower duty cycle reduces the power delivered. This method is efficient as the power electronics operate in a switching mode, minimizing power dissipation.

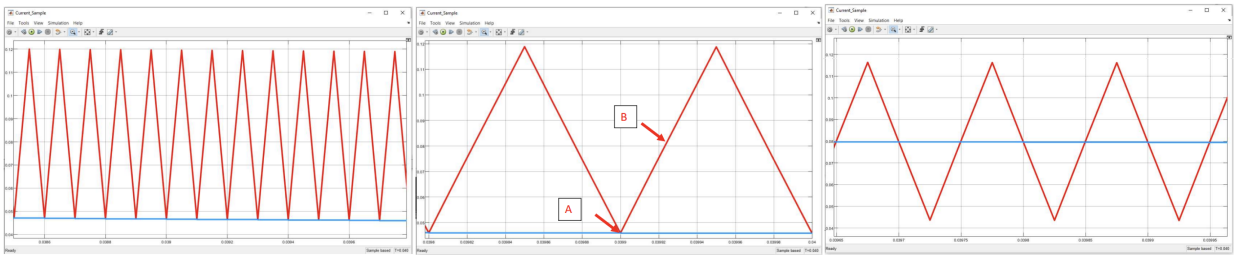
An initial experiment demonstrated that while low-frequency PWM causes the motor's speed to oscillate, high-frequency PWM (e.g., 10 kHz) results in a smooth rotation. This is because the motor's mechanical inertia acts as a natural low-pass filter, responding only to the average voltage.

However, the armature current is not filtered in the same way and exhibits a significant high-frequency triangular ripple. Attempting to measure this rippling current with a standard asynchronous ADC sample rate without matching frequencies of PWM and ADC, makes the measurement useless for control.

Samuele Pilati
Giorgio Giaretta
Tommaso Magelli



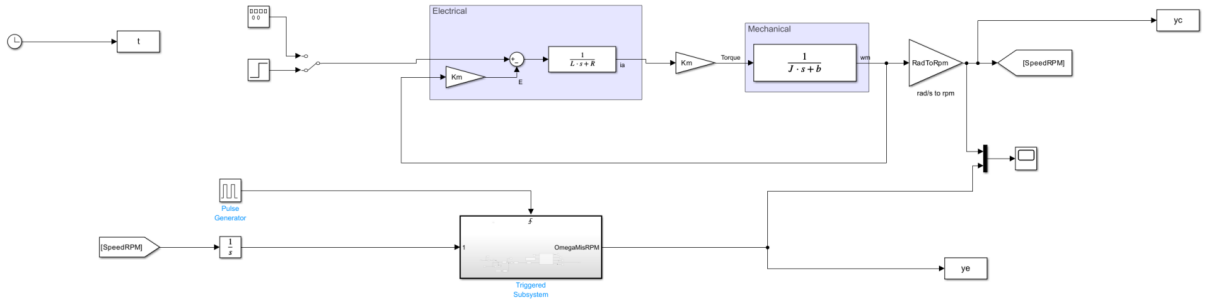
The solution implemented was to synchronize the ADC sampling with the PWM timer. By triggering the measurement at the precise midpoint of the PWM cycle, we can accurately capture the average current, effectively rejecting the high-frequency ripple.



ENCODER

We simulated the encoder to visualize the quantization error inherent in digital speed measurement. By comparing the ideal speed from the top model with the speed calculated by the simulated encoder, it becomes evident that the measured signal is not smooth. This effect is particularly pronounced at low speeds or with very high sampling rates, where an error of even a single count can lead to a significant percentage error in the final speed value. This simulation confirms the necessity of filtering the raw speed signal before using it for feedback, which was accomplished by implementing a low-pass filter in the subsequent laboratory exercises. This is why in all our models we'll use a filtered Omega signal.

Samuele Pilati
Giorgio Giaretta
Tommaso Magelli



CASCADE CONTROL

Theoretical Principle of Cascade Control

With the system interface established, the next step was to implement a robust feedback controller. For high-performance motor control, the common approach is a cascaded control structure. The theoretical basis for this architecture is the principle of dynamic separation, which involves designing nested control loops where each inner loop is significantly faster than the one enclosing it.

This methodology decouples the complex dynamics of the system into simpler problems. The innermost current loop is designed to be the fastest, making the motor behave like an ideal torque actuator. The outer velocity loop is designed to be slower and uses the inner loop as its actuator, simplifying its design to control a plant with primarily mechanical dynamics.

PI Controller Implementation in Simulink

For both the current and velocity loops, a Proportional-Integral (PI) controller was chosen. The integral action is essential for ensuring zero steady-state error in response to step commands. The continuous-time transfer function of the PI controller is:

$$R(s) = K_P(1+\frac{1}{T_i})$$

To implement this on the digital microcontroller, the controller was discretized using the Tustin (Bilinear Transformation) method. This results in the following difference equation, which was coded inside a MATLAB Function block in Simulink:

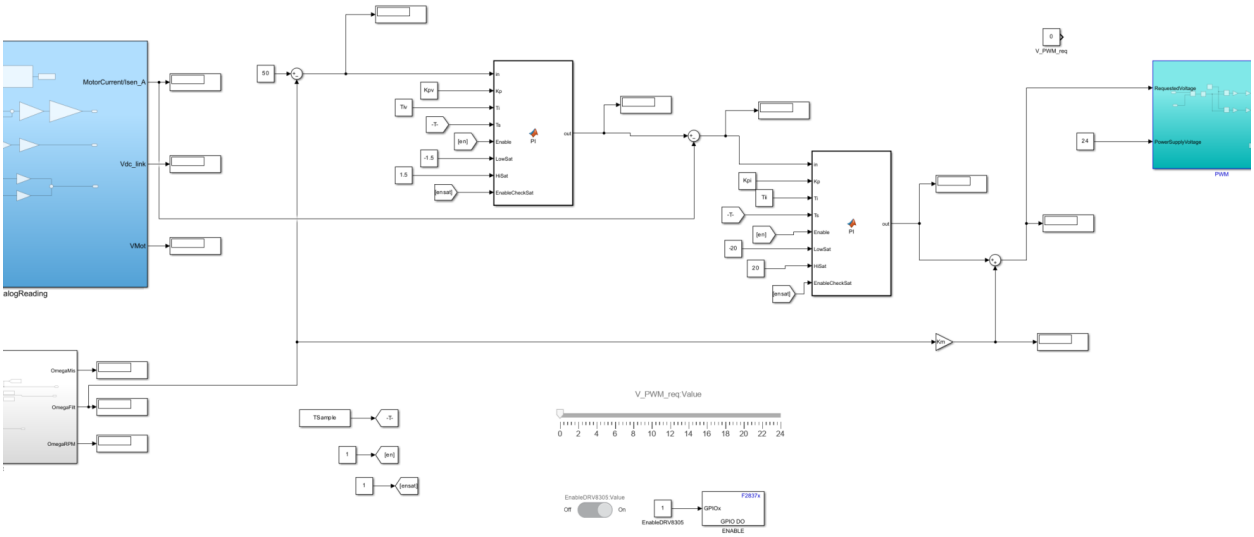
$$u(k) = K_p(e(k)-e(k-1)) + \frac{K_p T_s}{2T_i}(e(k)+e(k-1)) + u(k-1)$$

where u(k) is the controller output, e(k) is the error, and T_s is the sampling time.

```
function out  = PI( in ,Kp , Ti , Ts , Enable, LowSat, HiSat, EnableCheckSat)
persistent in_prev out_prev
% Initial Value
if isempty( out_prev )
    out_prev = 0;
end
if isempty( in_prev )
    in_prev = 0;
end
% G(s)=(Kp/Ti)*((1+Ti*s)/s) (Tustin)
% We run control only if motor is running
if (Enable == 1)
    %I Calculate the output
    out = Kp*Ts/(2*Ti)*(in+in_prev) + Kp*(in-in_prev) + out_prev;
else
    out = 0;
    in = 0;
end

if(EnableCheckSat == 1)
        %check for saturation
        if out > HiSat
            out = HiSat;
        elseif (out < LowSat)
            out = LowSat;
        end
end
    out_prev=out;
    in_prev=in;
end
```

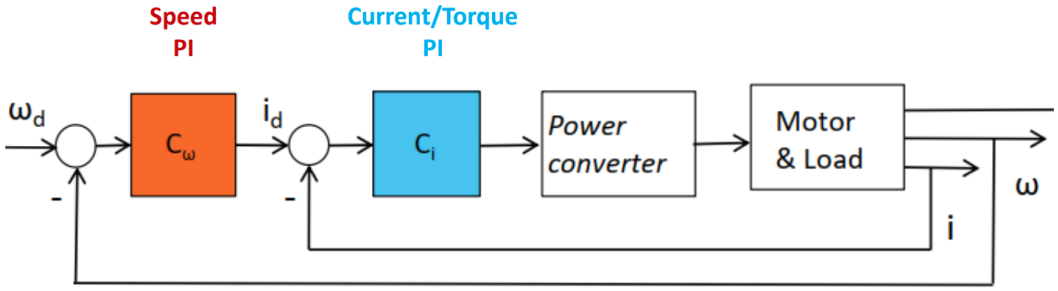
Each PI controller block was also equipped with an Enable input, allowing the control action to be activated or deactivated during testing.



Samuele Pilati
Giorgio Giaretta
Tommaso Magelli

2 Controller Tuning and BEMF Compensation

The tuning of the PI parameters (K_p and T_i) for each loop was performed systematically using the pole-zero cancellation technique, based on the motor parameters identified in the previous system identification phase.



- **Current Loop Tuning:** The design of the inner loop is consistent across all methods. The goal was to make it fast and stable. To achieve this, the integral time constant (T_{ii}) was set equal to the motor's electrical time constant, cancelling the electrical pole:

$$T_{ii} = \tau_e = \frac{L}{R}$$

The proportional gain (K_{pi}) was then calculated to achieve a desired fast closed-loop time constant of τ_i=1 ms. The gain is calculated with the formula:

$$K_{pi} = \frac{L}{\tau_i}$$

- **Velocity Loop Tuning:** For the outer velocity loop, a PI controller was also used ($R\omega(s)$), with its integral action ($T_i\omega$) set to cancel the dominant mechanical pole ($T_i\omega = \tau_m = \frac{J}{b}$). However, for tuning the proportional gain ($K_p\omega$), two theoretical approaches were considered.

    1. **Full Second-Order Model Approach:** The first method considers the full dynamics of the plant, including the first-order behavior of the current loop. The resulting closed-loop transfer function for the velocity is a second-order system:

$$F_\omega(s) = \frac{\omega_n^2}{s^2 + 2\delta\omega_n s + \omega_n^2}$$

In this case, tuning involves choosing a desired damping ratio, δ, and then calculating the single gain $K_p\omega$ that achieves it. While theoretically complete, this approach is less intuitive because the final response speed (related to ω_n) is a consequence of the choice of δ, rather than a direct design parameter.

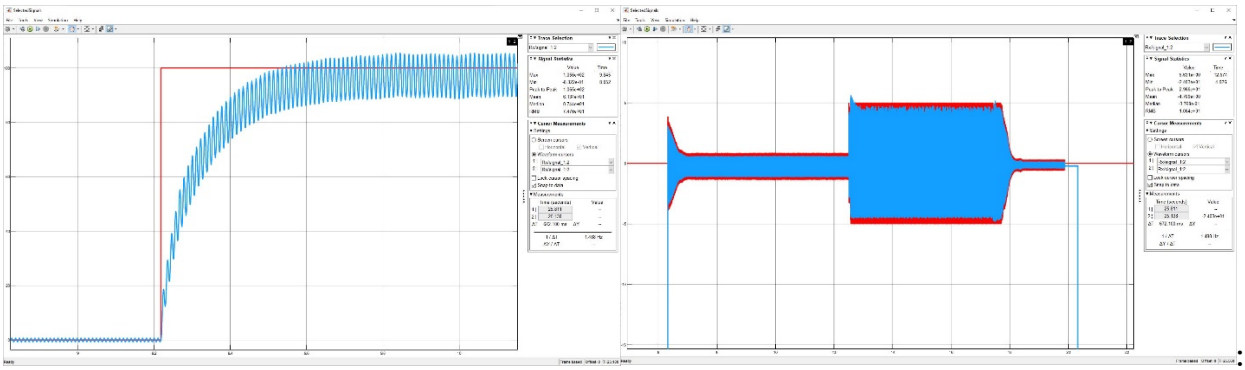
2. **Simplified First-Order Model Approach (Chosen Method):** The second approach, which we adopted for our implementation, leverages the principle of dynamic separation. Since we designed the current loop to be much faster than the mechanical dynamics ($\tau_i \ll \tau_m$), we can simplify our design by approximating the current loop as instantaneous. This simplifies the velocity closed-loop system to a first-order model:

$$F_\omega(s) \approx \frac{1}{1+\tau_\omega s}$$

We selected the first-order approximation because it offers a more direct and practical tuning methodology. It allows us to define the desired performance with a single parameter: the closed-loop time constant τ_ω. In our MATLAB script, we chose the target time constant to be ten times slower than that of the current loop ($\tau_\omega = 10 \cdot \tau_i = 10\text{ms}$). The proportional gain ($K_{p\omega}$) was then calculated directly to achieve this response time using the formula:

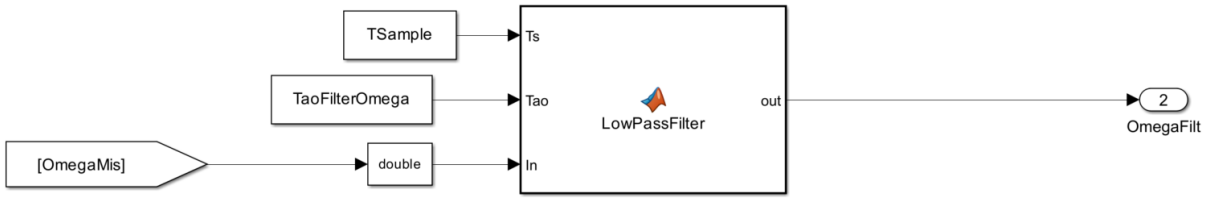
$$K_{p\omega} = \frac{J}{\tau_\omega K_m}$$

To validate the importance of a correct tuning of dynamic parameters we simulated the behavior of the step response of both the current and velocity loop with different parameters than the calculated (ideal) ones, these are some examples:



Samuele Pilati
Giorgio Giaretta
Tommaso Magelli

FILTER




```
function out  = LowPassFilter(  Ts ,Tao, In)

persistent OutPrecedente
if isempty( OutPrecedente )
    OutPrecedente = 0 ;
end
    %Filtro passa basso discretizzato con il metodo delle differenze
    %all'indietro

    a=Ts/(Ts+Tao);
    b=Tao/(Ts+Tao);

    out=a*In+OutPrecedente*b;

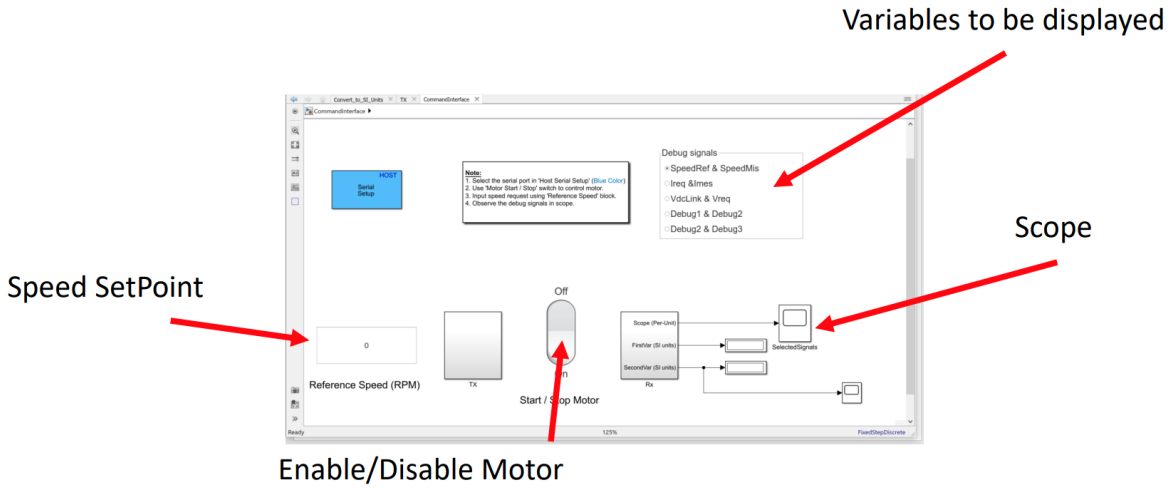
    OutPrecedente = out;

end
```

We are filtering the high frequency noise in the encoder measured speed to work with cleaner input signals in our control.

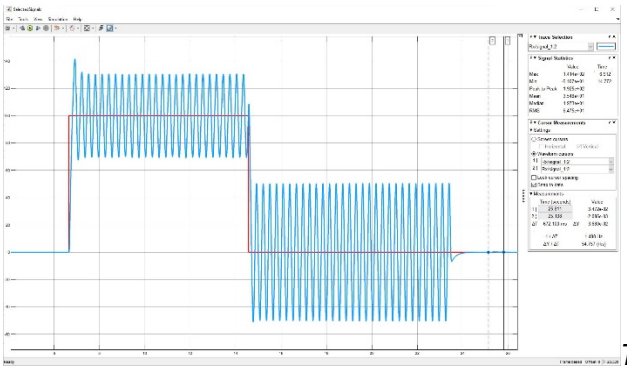
TRACING

Before we were given the command interface, we were uncapable to plot data while we were connected to the motor.

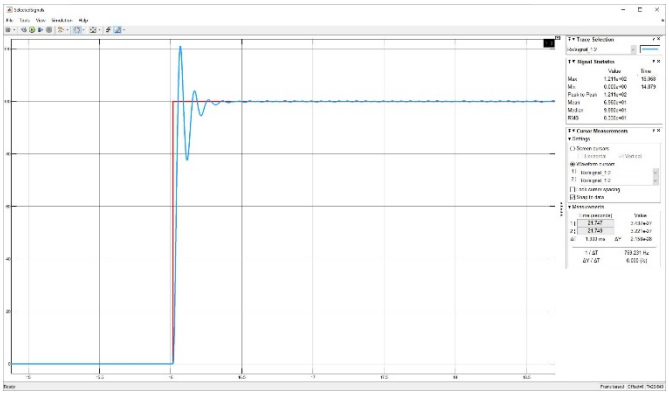


Thanks to communication blocks we are now able to run the motor for a window of time and, once the data are saved, decide which of them we want to plot.

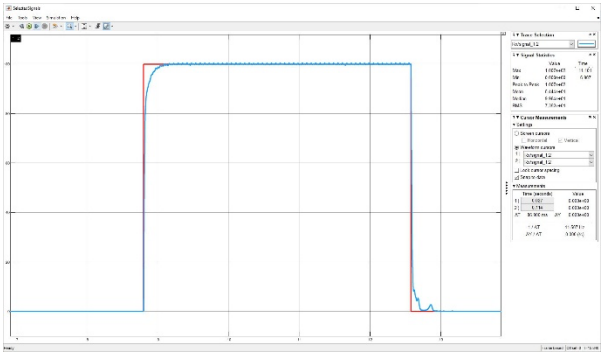
One of the first real approaches to this interface has been done trying to saving plots of different responses given changing the value of tau omega filter for the low pass filter getting these results:



TaoFilterOmega = 0.2;



TaoFilterOmega = 0.02;



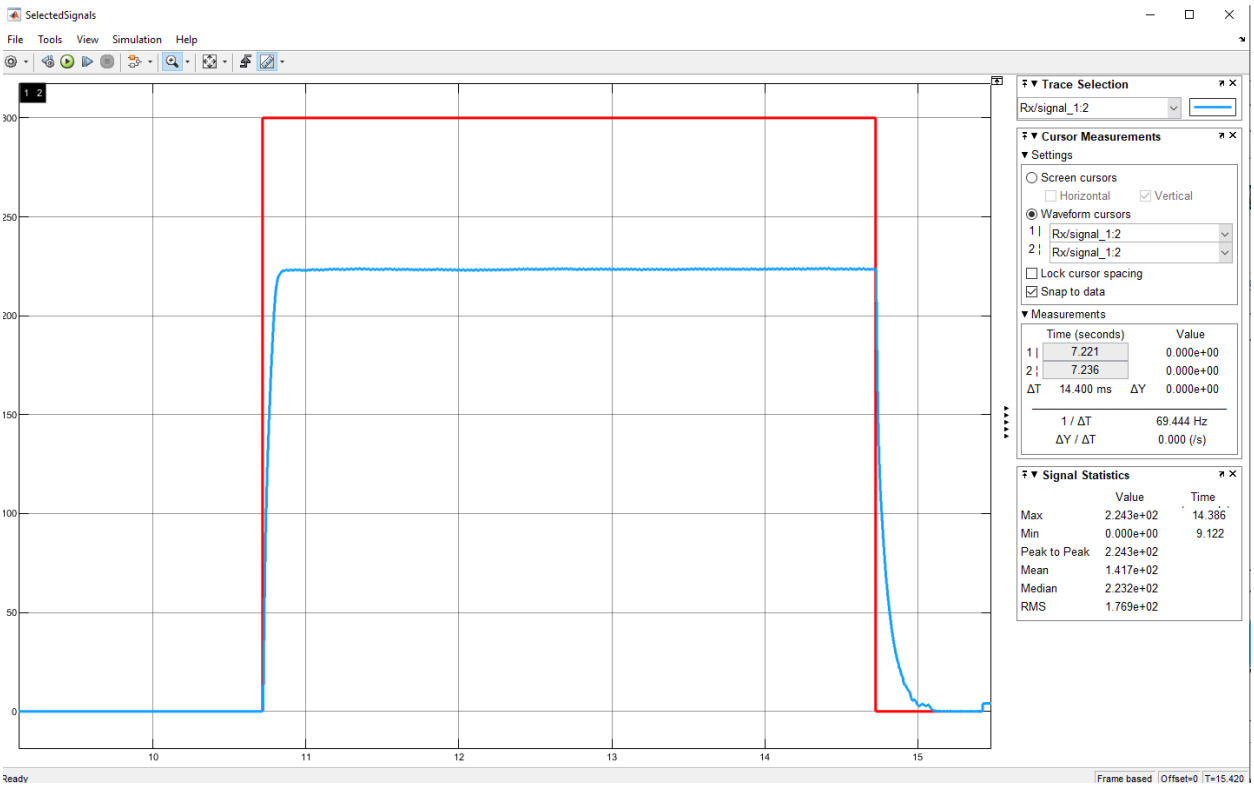
TaoFilterOmega = 0.002;

IMPLEMENTATION OF THE ANTI-WINDUP SCHEME

To prevent the integrator windup phenomenon, which can cause large overshoots and system instability, a simple but effective anti-windup scheme was implemented. The logic was coded directly within the PI controller's MATLAB Function block. This scheme works by taking the controller's calculated output: if the computed value exceeds the predefined saturation limits (HiSat and LowSat), it is limited to that maximum or minimum value before being sent to the motor driver and recorded for the next control step.

The effectiveness of this implementation is demonstrated in the test below, where a reference command exceeds the system's physical capabilities. The Anti-windup action can be noticed clearly because when the reference input is set back to 0 rad/s, although the

system's response was completely saturated, the system behaves instantly in a correct way, by "instantly" decreasing its speed.



TRAJECTORY TRACKING AND FEEDFORWARD CONTROL

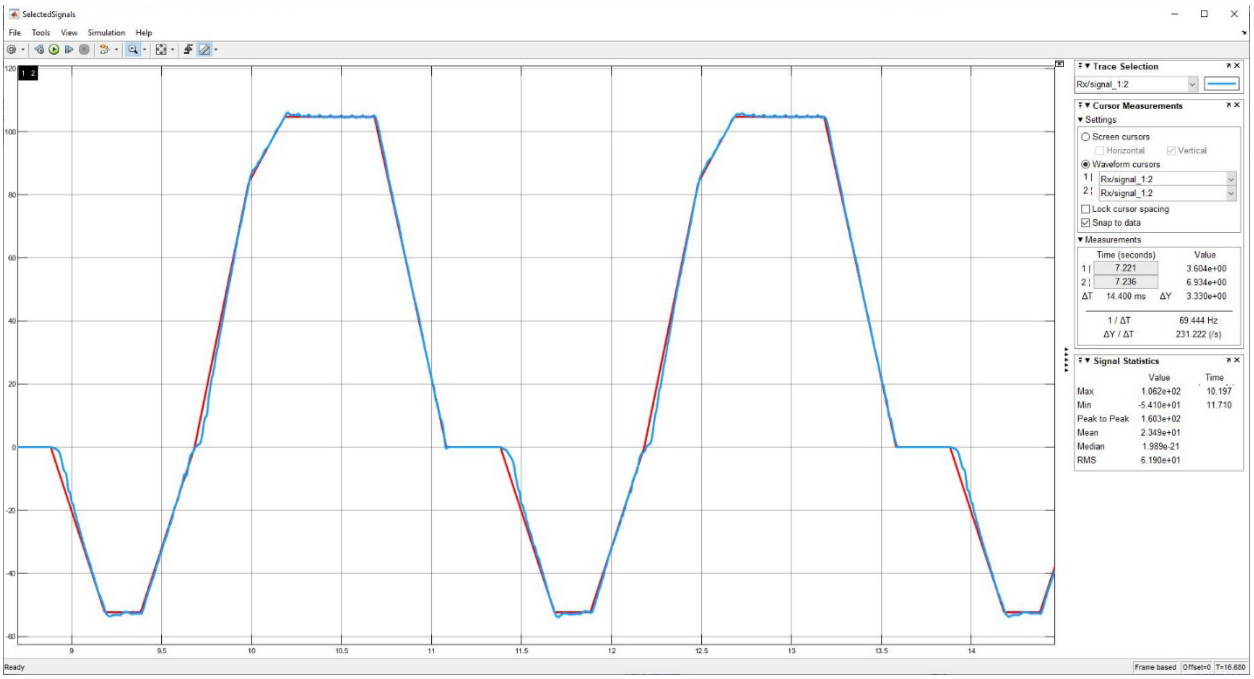
1. Trajectory Tracking

While the cascaded PI controller demonstrated effective regulation to a constant setpoint, some applications require following a time-varying trajectory. The objective of this laboratory session was to evaluate the performance of our previously tuned PI controller in a trajectory tracking scenario and to implement a feedforward controller to improve its accuracy.

The reference signal used for these tests was a trapezoidal velocity profile, which involves phases of constant acceleration, constant velocity, and constant deceleration.

2. Limitation of Pure Feedback Control

The initial test involved commanding the motor to follow the trapezoidal velocity profile using only the PI feedback controller. The results immediately highlighted a limitation of the control: the presence of a noticeable tracking error on steep trajectory changes.



This error occurs because a feedback controller can only generate a corrective action after an error has been measured. To produce the necessary torque for acceleration, a persistent velocity error is required to drive the PI controller's output.

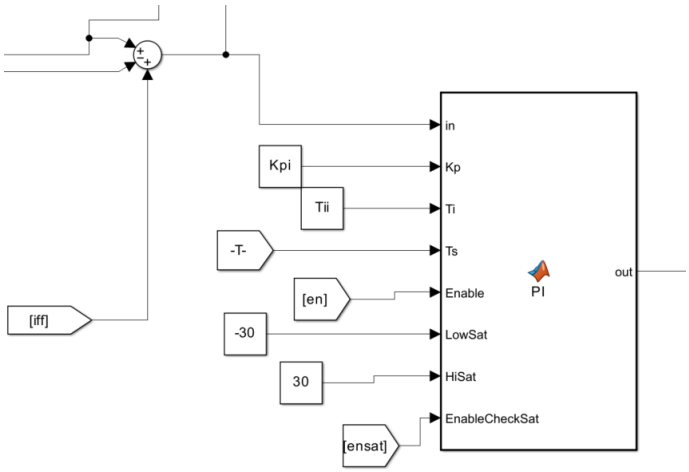
3. Feedforward Control Implementation

To overcome this limitation, a feedforward control action was implemented. The principle of feedforward is to use the known reference trajectory to proactively calculate the control effort required to follow it.

- **Theoretical Basis:** The required torque for a given acceleration is dictated by the motor's mechanical dynamics, primarily its inertia (J). Neglecting friction, the equation is $C_m(t) \approx J\omega'(t)$. Since torque is proportional to current ($C_m = K_m i$), we can pre-calculate the current needed to produce the desired acceleration $\omega'd$:

$$i_{ff} = \frac{J}{Km}\omega'd$$

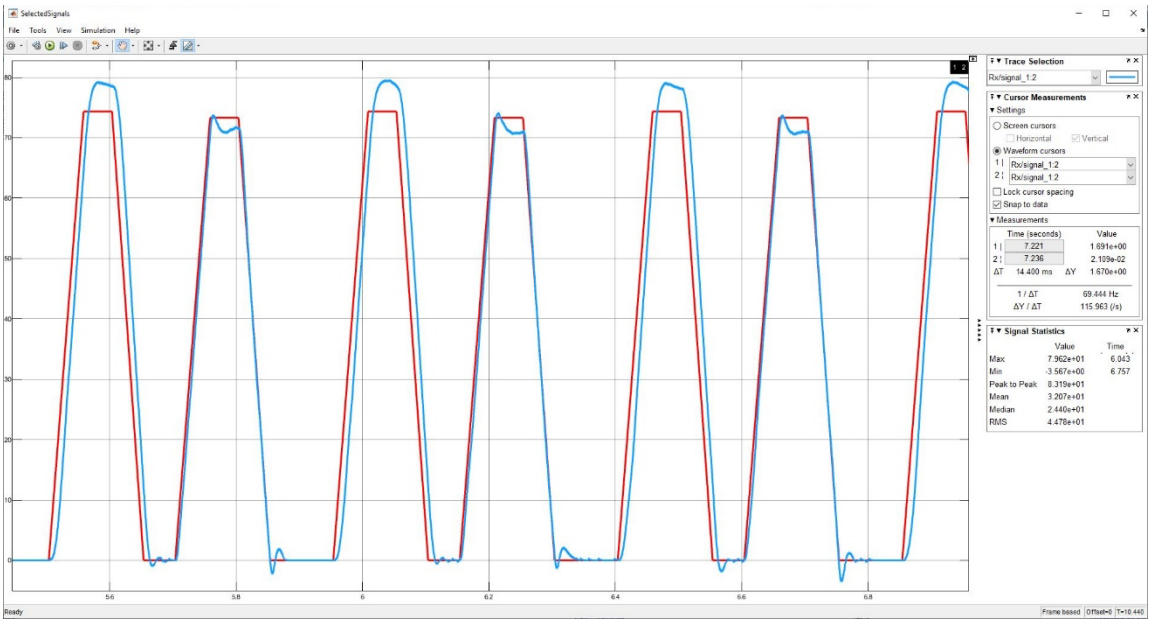
- **Implementation:** In Simulink, the acceleration profile was derived from the reference velocity trajectory. The feedforward current, i_{ff}, was calculated using the formula above with the motor parameters. This feedforward command was then summed directly with the output of the PI velocity controller, as shown in the diagram below.



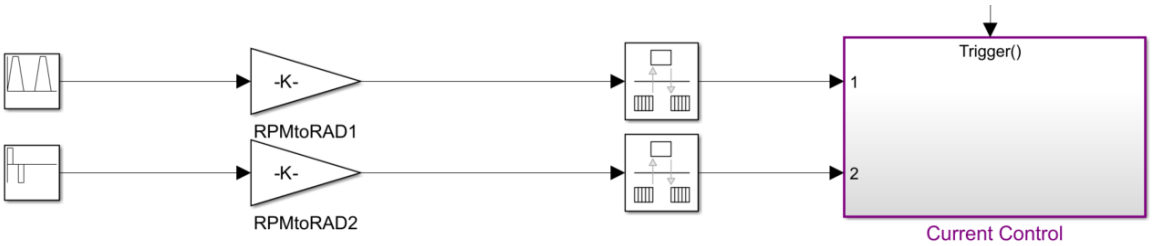
Block diagram of the control scheme with the addition of the acceleration feedforward path.

5.4. Results and Analysis

The effect of adding the feedforward controller was evident:



Comparison of velocity tracking error with and without feedforward control. To better see the effect of FF we want to implement once every two trapezoids.



The feedback PI controller is then left with just the task of correcting for minor model inaccuracies and external disturbances. This experiment demonstrates that for high-performance trajectory tracking, a combination of both feedback and feedforward control is essential.

Feedforward control based on inverting the system's model has practical limitations. It is only feasible if the model is minimum-phase and has a relative degree of zero; otherwise, the resulting controller would be unstable or non-causal.

In the lab, this issue was avoided by using a pre-calculated trajectory where the necessary acceleration values were already known.

SYSTEM IDENTIFICATION VIA LEAST SQUARES ESTIMATION

1. Objective and Theoretical Principle

While the manufacturer's datasheet provides a valid starting point for system modeling, these nominal parameters often deviate from the real-world behavior of a specific motor due to manufacturing tolerances and operating conditions. To achieve a higher level of control performance, it is essential to work with a model that accurately reflects the true dynamics of our plant.

The objective of the following laboratory session was to experimentally determine the physical parameters of our DC motor. The chosen method was the Least Squares (LS) algorithm, a powerful technique that estimates the parameters of a model by minimizing the sum of squared errors between the model's predicted output and the actual measured output.

To apply this method, the system dynamics were first discretized using the backward difference method, resulting in a linear regression model of the form:

$$y(k) = \alpha_1 * y(k-1) + \alpha_2 * u(k)$$

where the coefficients α_1 and are α_2 directly related to the physical parameters we wish to find.

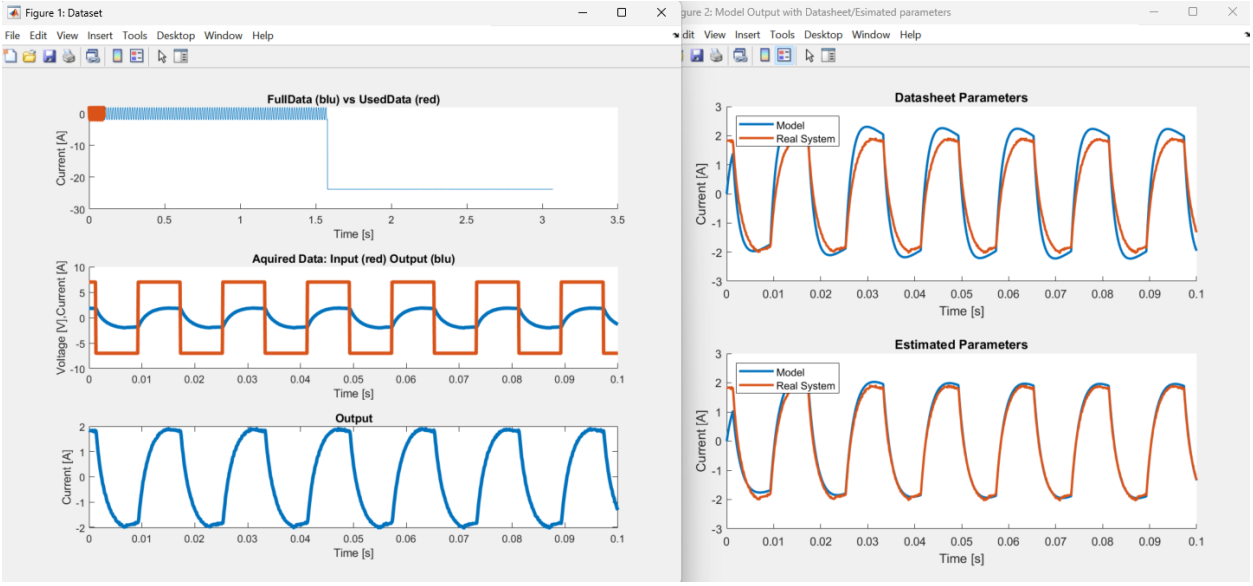
2. Experimental Procedure

To simplify the identification process, the motor's electrical and mechanical dynamics were decoupled and estimated in two separate experiments.

1. Estimation of Electrical Parameters (R and L):

- **Procedure:** To isolate the electrical dynamics, the back-EMF term was eliminated by keeping the motor stationary.
- **Analysis:** The acquired data was processed by a MATLAB script implementing the LS algorithm to find the coefficients of the discrete-time model for the electrical subsystem. These coefficients were then used to calculate the armature resistance (R) and inductance (L).

$$L_{est} = \frac{\alpha_1}{\alpha_2} T_{sample} = 0.0063 \qquad R_{est} = \frac{1-\alpha_1}{\alpha_2} = 3.4604$$

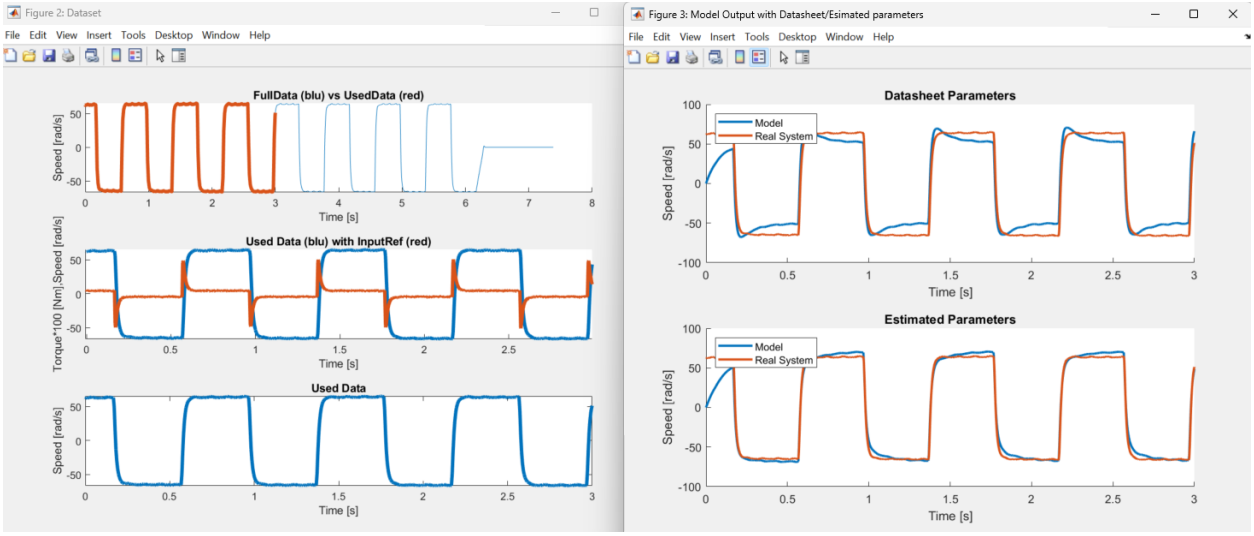


Input voltage and output current data acquired for the estimation of electrical parameters (R and L).

2. Estimation of Mechanical Parameters (J and b):

- **Procedure:** To identify the mechanical parameters, the motor was at an alternating positive-negative speed. The input torque (calculated from the measured current, as $C_m = K_m i$) and the output angular velocity (ω) were logged.
- **Analysis:** The LS algorithm was applied to this second set of data to estimate the coefficients of the mechanical subsystem's discrete model. These were then used to calculate the total rotor inertia (J) and the viscous friction coefficient (b).

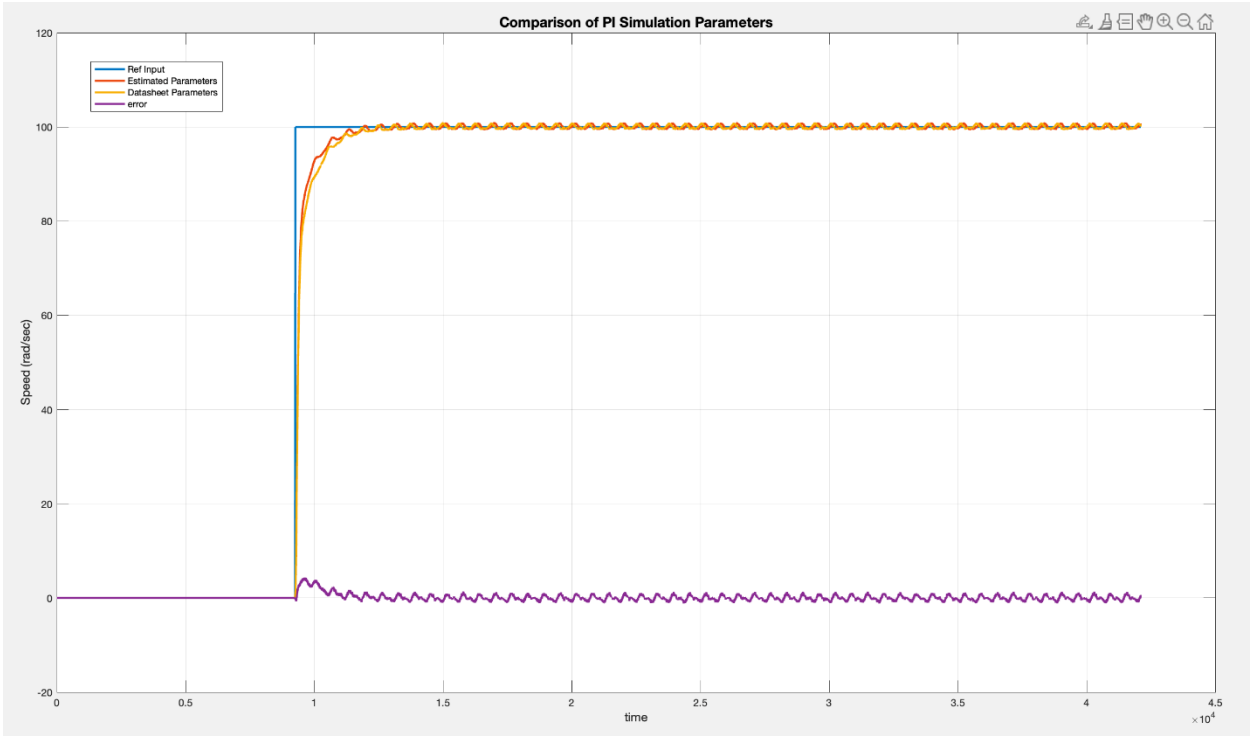
$$b_{est} = \frac{1-\alpha_1}{\alpha_2} = 6.332 \times 10^{-4} \qquad J_{est} = \frac{\alpha_1}{\alpha_2} T_{sample} = 7.913 \times 10^{-5}$$



Input torque and output speed data acquired for the estimation of mechanical parameters (J and b).

5.3. Results and Validation

The most important validation step was to compare the performance of the PI velocity controller when tuned with the two different sets of parameters.



Although the difference among the two responses isn't great, the results showcase that the controller tuned with the experimentally identified parameters provides a slightly faster response. In addition it's noticeable how the maximum error (difference) between the two

signals is of only 4 rad/s at the startup, and that at steady state the ripple (error) is lower than 1% and coherent with the displacement of the driven motor's axes. This validates the system identification process and demonstrates its importance for achieving high-performance control.

ANALYTICAL DESIGN WITH A DEADBEAT CONTROLLER

1. Theoretical Principle and Design

To explore alternatives to the classic PI controller, we first implemented a Deadbeat controller. This is a discrete-time analytical design technique whose primary objective is to achieve the fastest possible response to a setpoint change. Theoretically, it forces the system's output to reach and settle at the reference value in the minimum possible number of sampling periods, which is determined by the order and intrinsic delay of the plant.

The design was based on the simplified method, where the desired closed-loop transfer function is set to a pure time delay, $G_m(z) = z^{-k}$. This leads to a direct formula for the controller: $D(z) = \frac{1}{G_p(z)} \cdot \frac{z^{-k}}{1 - z^{-k}}$

The controller was synthesized in a MATLAB script using the estimated plant model, Gp(z), and then implemented in Simulink as a discrete transfer function block.

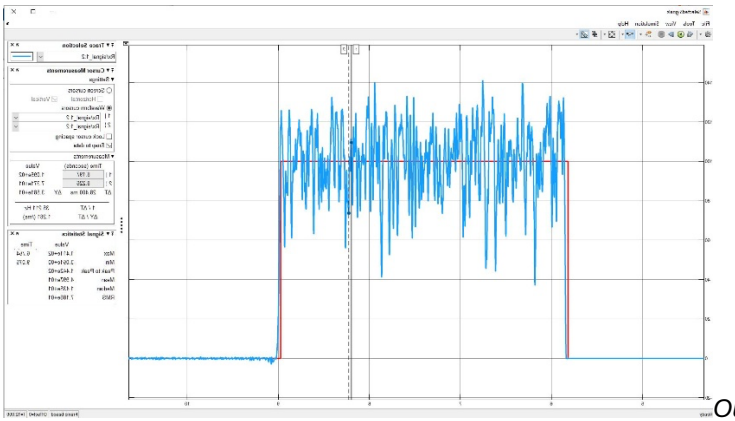
2. Anti-Windup Implementation

The Deadbeat controller is known for its highly aggressive control action, making it extremely susceptible to actuator saturation and the resulting integrator windup problem. To manage this, a more sophisticated anti-windup scheme than simple clamping was implemented, as introduced in the theory and lab guide.

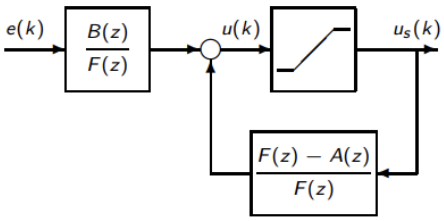
This structure uses a polynomial, F(z), which acts as a dynamic filter within the anti-windup feedback path.

The placement of the poles in this stable polynomial enables us to choose the controller's behavior after saturation, so that we can balance the speed of recovery with its smoothness and the degree of overshoot.

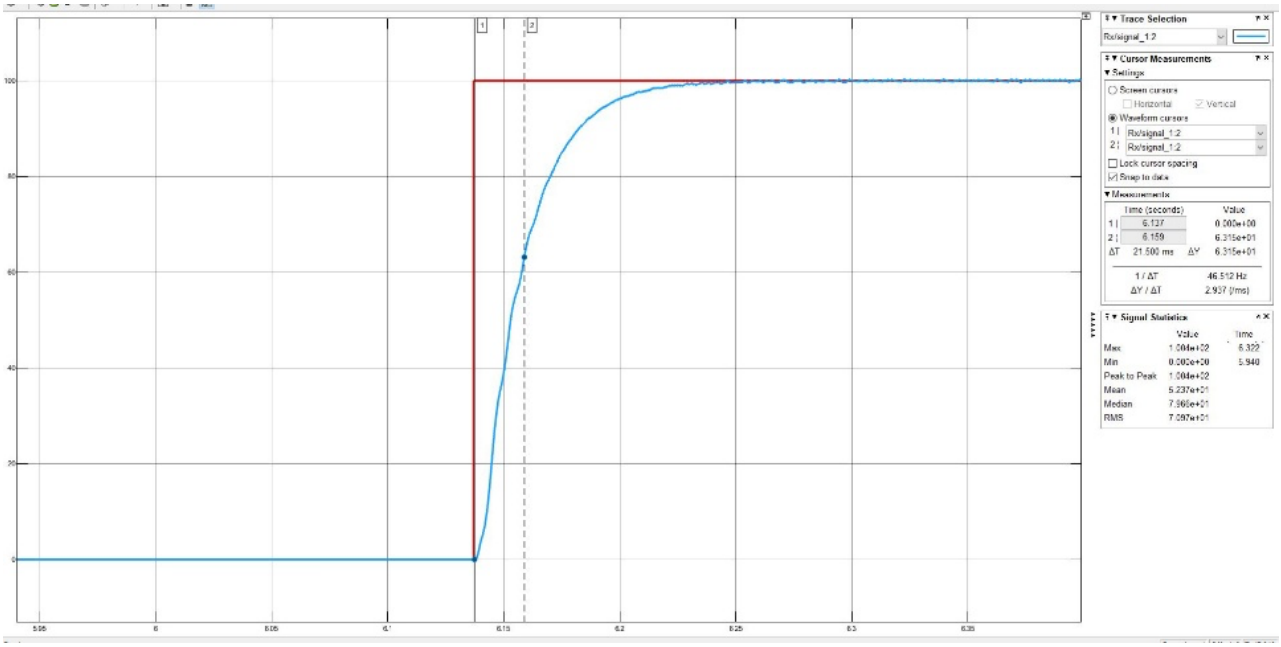
Samuele Pilati
Giorgio Giaretta
Tommaso Magelli



Output of speed without anti-windup scheme

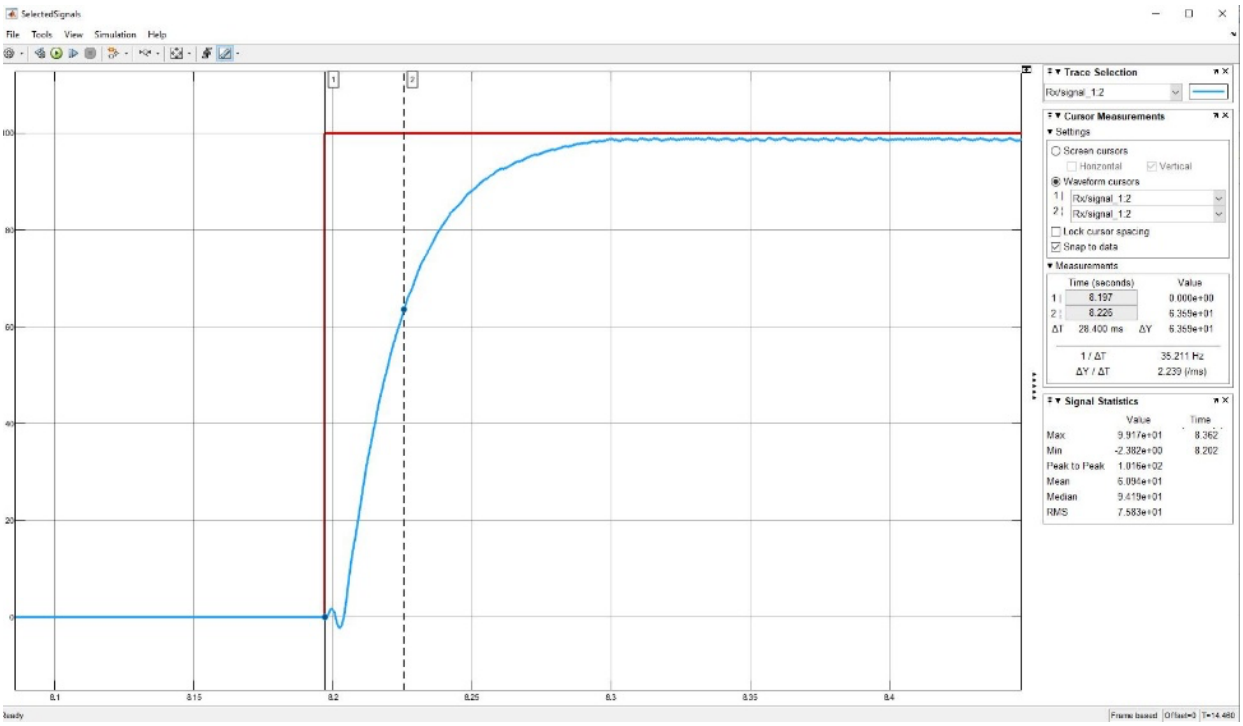


Block diagram of the implemented anti-windup scheme

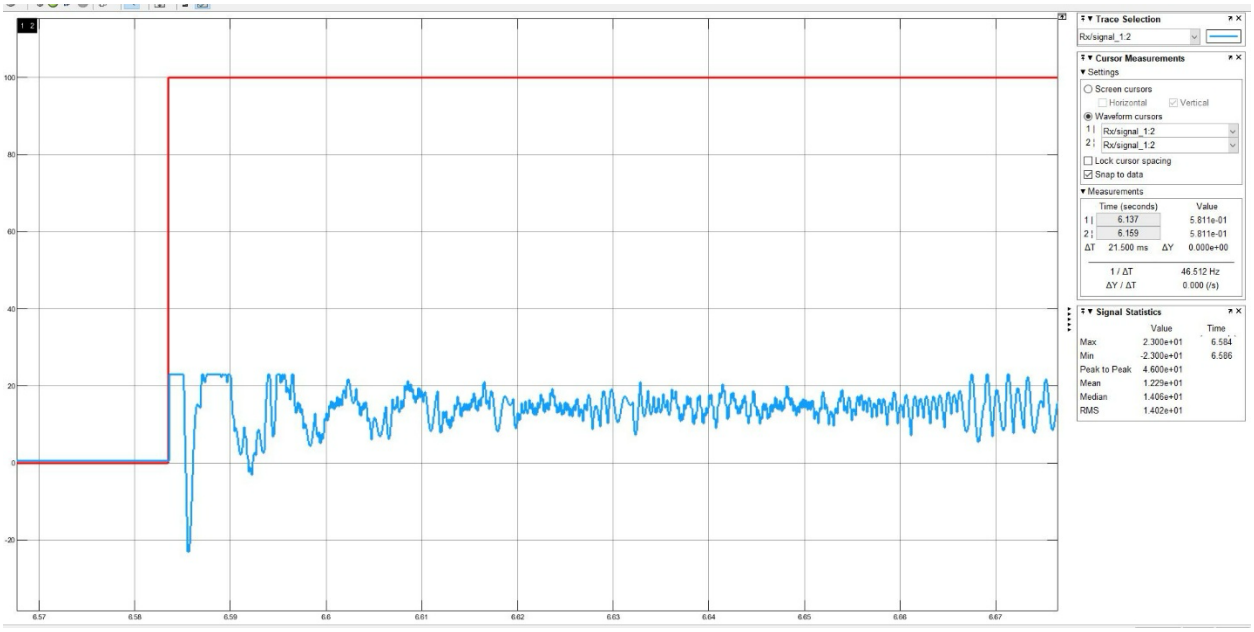


Output of speed with anti-windup scheme and no load – time constant 21.5ms

Samuele Pilati
Giorgio Giaretta
Tommaso Magelli



Output of speed with anti-windup scheme with load – time constant 28.4ms



Voltage request with anti-windup scheme and no load

Samuele Pilati
Giorgio Giaretta
Tommaso Magelli

ANALYTICAL DESIGN WITH A DAHLIN CONTROLLER

1. Theoretical Principle and Design

As an alternative to the aggressive nature of the Deadbeat controller, we implemented a Dahlin controller. This analytical design method is also based on specifying a desired closed-loop behavior, but instead of the fastest possible response, it aims for a smoother, more predictable one.

The core principle of the Dahlin controller is to make the closed-loop system respond like a first-order system with a time delay. The desired response to a step input is defined by the following continuous-time transfer function: $Y(s) = \frac{e-\theta s}{\lambda s+1}\cdot\frac{1}{s}$

The key design parameter is λ (lambda), which represents the desired time constant of the closed-loop system. This parameter provides an intuitive tuning tool:

- A small λ results in a faster response.
- A large λ results in a slower, more damped response.

After discretizing this desired model to get Gm(z), the controller D(z) is calculated using the same analytical formula as before:

$$D(z) = \frac{1}{G_p(z)}\cdot\frac{G_m(z)}{1-G_m(z)}$$

2. Implementation and Tuning

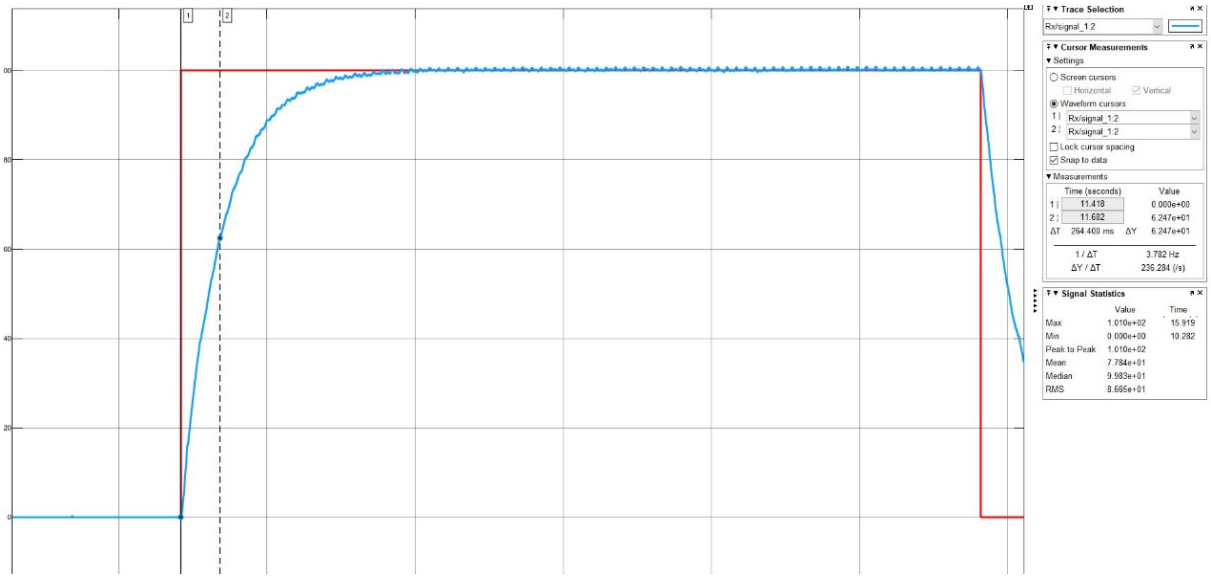
The controller was designed in a MATLAB script, where the desired time constant λ was set as a tuning parameter. The script calculated the numerator and denominator coefficients of the resulting controller, D(z). These coefficients were then loaded into a Discrete Transfer Function block in Simulink, which implemented the control logic on the hardware. As with the Deadbeat controller, this was integrated with the advanced anti-windup scheme to ensure robust performance in the presence of actuator saturation.

3. Experimental Results and Analysis

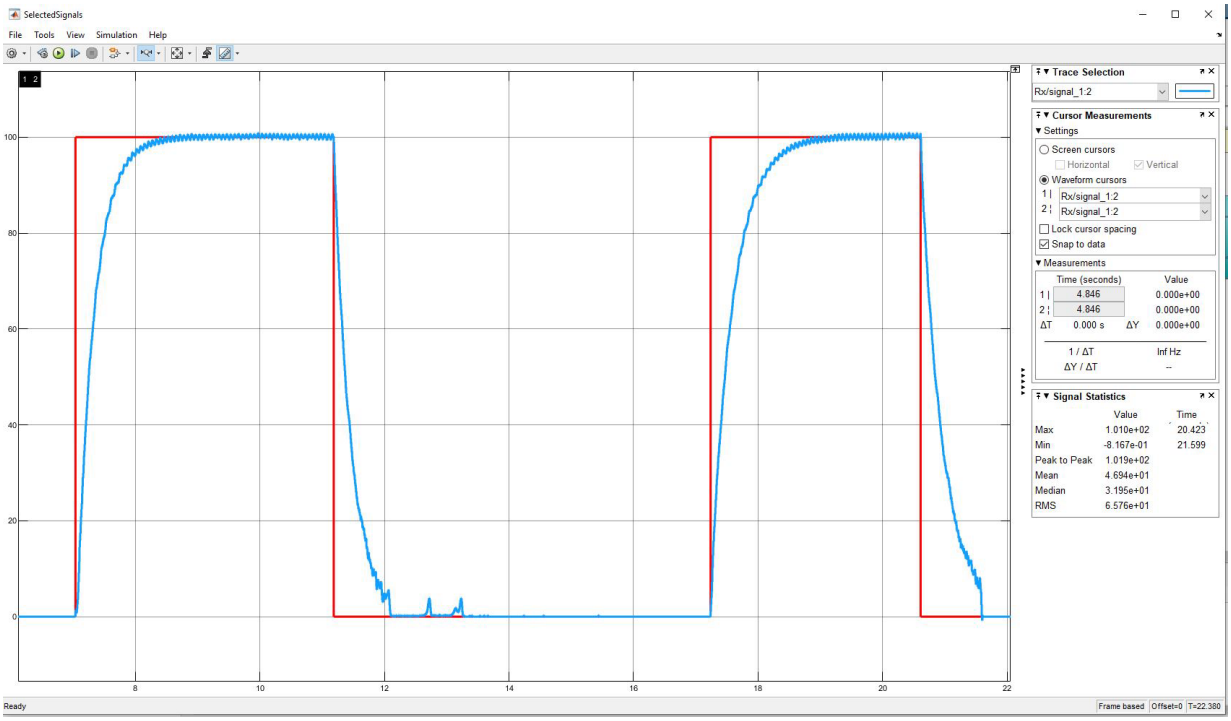
The primary experiment involved testing the system's response to a step command using different values for the tuning parameter λ.

Setting λ to 0.2:

Samuele Pilati
Giorgio Giaretta
Tommaso Magelli

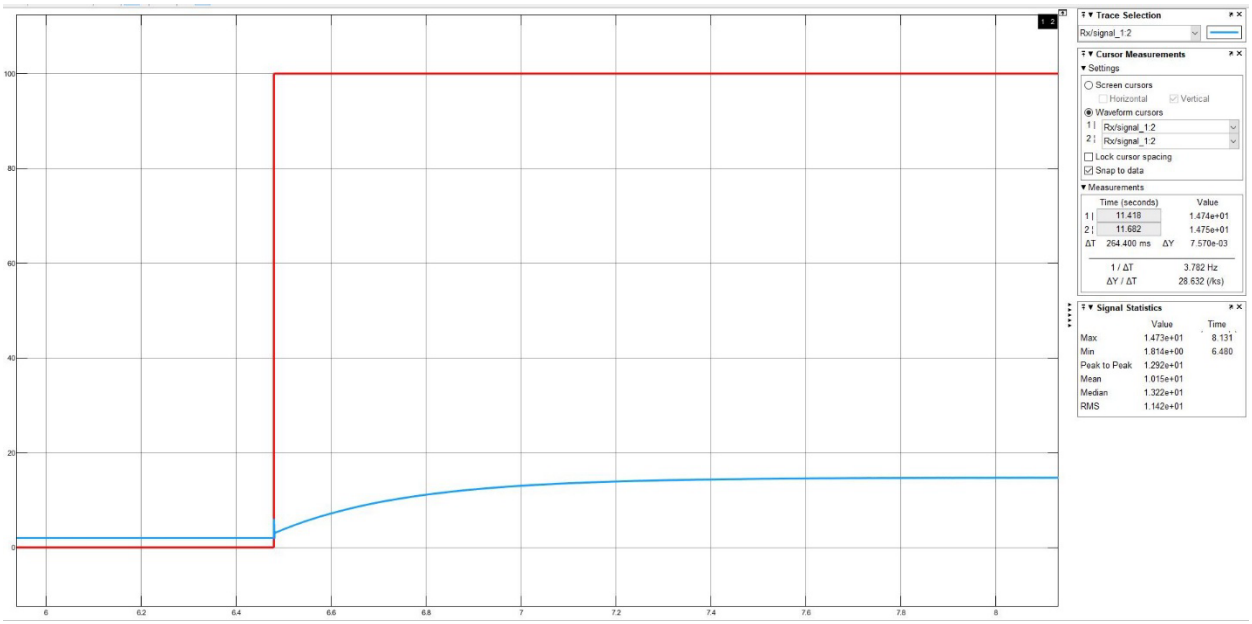


Dahlin controller performance for tuning parameter λ=0.2 *and time constant = 264ms*

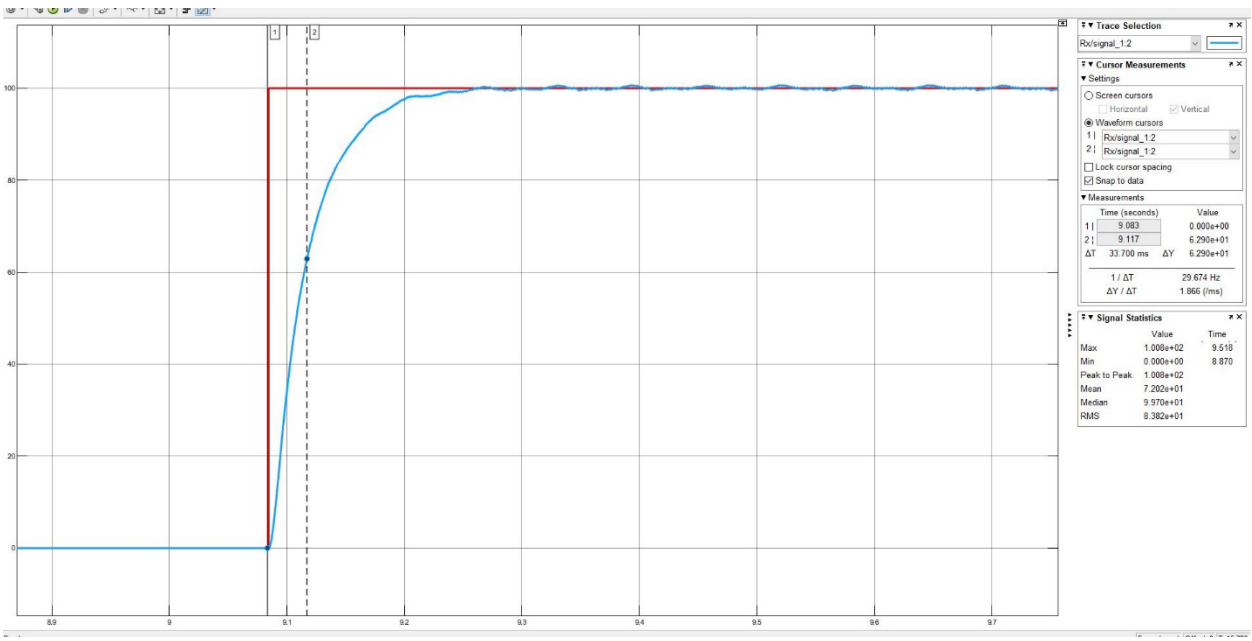


Dahlin controller performance for tuning parameter λ=0.2 with and without load

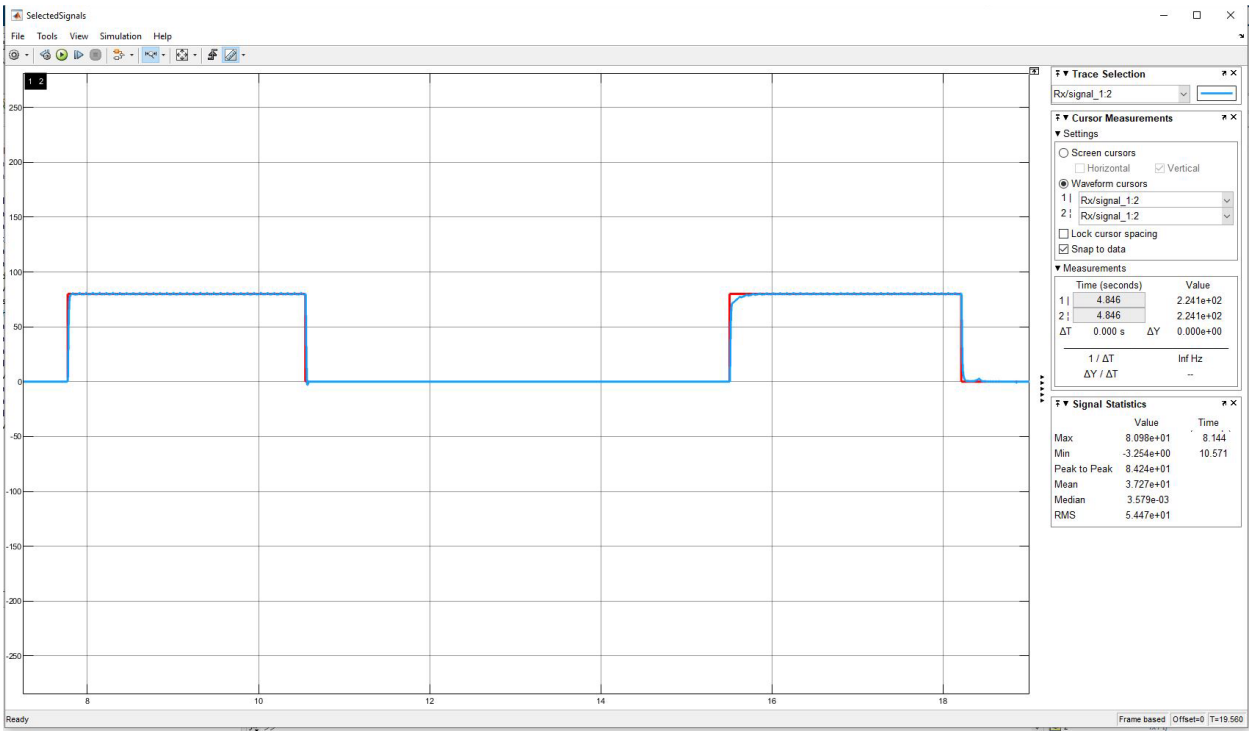
And it's control effort



Later, decreasing the λ to 0.02:

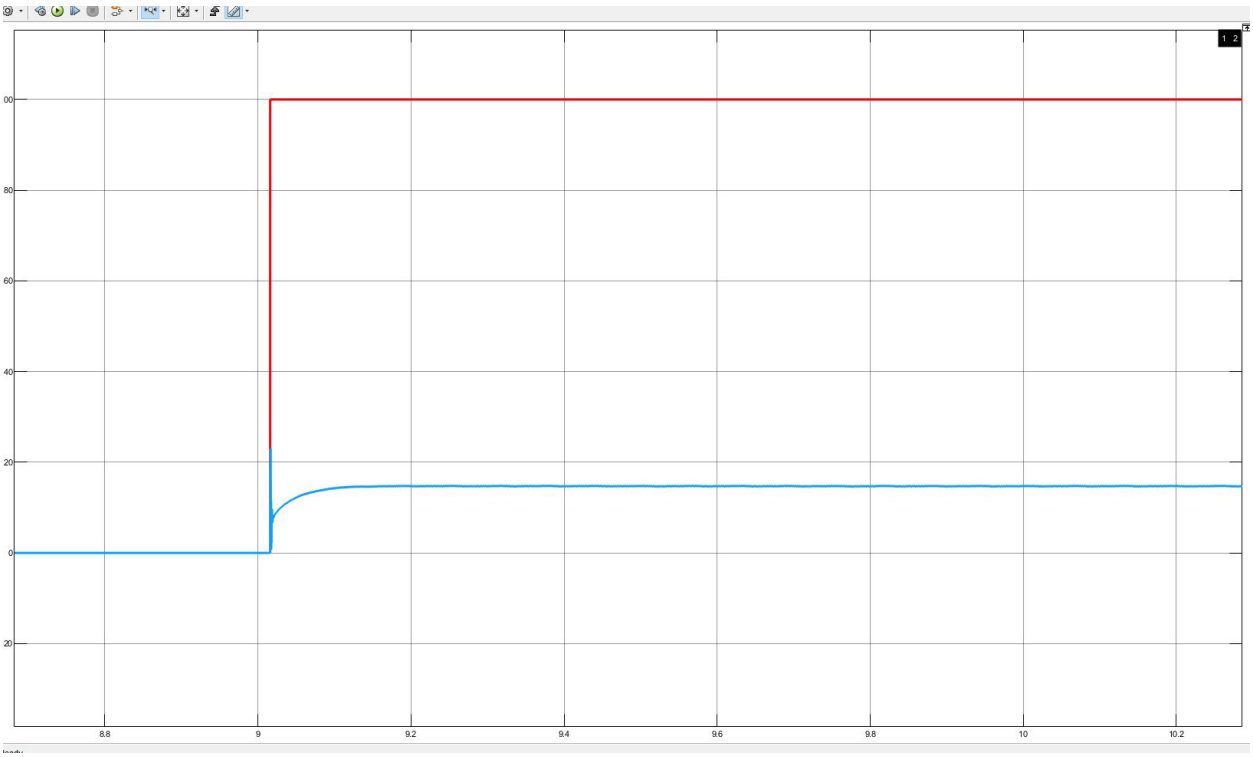


Dahlin controller performance for tuning parameter λ=0.02 *and time constant = 33.7ms*



Dahlin controller performance for tuning parameter λ=0.02 with and without load

And it's control effort



The results clearly demonstrate the effect of λ as the main tuning parameter. It provides direct and intuitive control over the trade-off between the speed of response and the smoothness of the control action. Compared to the Deadbeat controller, the Dahlin design allows a less aggressive response, that leads to less control effort and a smother trajectory.

(BONUS) ONLINE PARAMETERS ESTIMATION

1. Theoretical Principle and Design

Other than the experiments conducted in the lab, we attempted implementing an online parameter estimator.

The recursive estimation process allows a controller to update its understanding of a system in real time, one data sample at a time. This method is crucial for adaptive control, where system parameters might change over time.

The Recursive Least Squares (RLS) Algorithm

The standard least squares formula is not suitable for real-time control because it requires a complete set of historical data to be processed all at once. The recursive formulation solves this by starting with an initial guess and incrementally refining the parameter estimates as new data arrives.

The core of the Recursive Least Squares (RLS) algorithm is the update equation:

$$\hat{\alpha}(N+1) = \hat{\alpha}(N) + k(N+1) * [y_{N+1} - \gamma^T \hat{\alpha}(N)]$$

This can be broken down as:

- **New Estimate ($\hat{\alpha}$(N+1)):** The updated parameter vector.
- **Old Estimate ($\hat{\alpha}$(N)):** The best guess from the previous time step.
- **Correction Term:** The second part of the equation, which adjusts the old estimate. It consists of:
    - **Prediction Error ($y_{N+1} - \gamma^T * \alpha(N+1)$)**The difference between the new measured output (yN+1) and the output predicted by the old model. This error is the "new information" that drives the adaptation.

- **Correction Gain (k(N+1)):** A vector that scales the prediction error, determining how much the estimate should be adjusted. This gain is also updated recursively

We also add a Forgetting factor to allow the estimation to change easily even after a significant amount of time.

The equations we implement are the following:

$$\begin{cases} \hat{\alpha}(N+1) &= \hat{\alpha}(N) + k(N+1)[y_{N+1} - \gamma^T \hat{\alpha}(N)] \\ k(N+1) &= P(N)\gamma[\beta + \gamma^T P(N)\gamma]^{-1} \\ P(N+1) &= \frac{1}{\beta}[I_n - k(N+1)\gamma^T]P(N) \end{cases}$$

2. Experimental Results and Analysis

```
function alpha1 = fcn(uprev, y, yprev,beta,en)

persistent p k alpha c

if isempty(p )
    p=eye(2);
end

if isempty(k )
    k=zeros(2,1);
end

if isempty(alpha )
    alpha=[1;0.1161];
end

if isempty(c )
    c=0;
end
if(en)
if (c>2)
lambda=[yprev;uprev];

k=p*(lambda/(beta+transpose(lambda)*p*lambda));
p=((eye(2)-k*transpose(lambda))/beta)*p;
alpha=alpha+k*(y-transpose(lambda)*alpha);
else
    c=c+1;
end
end

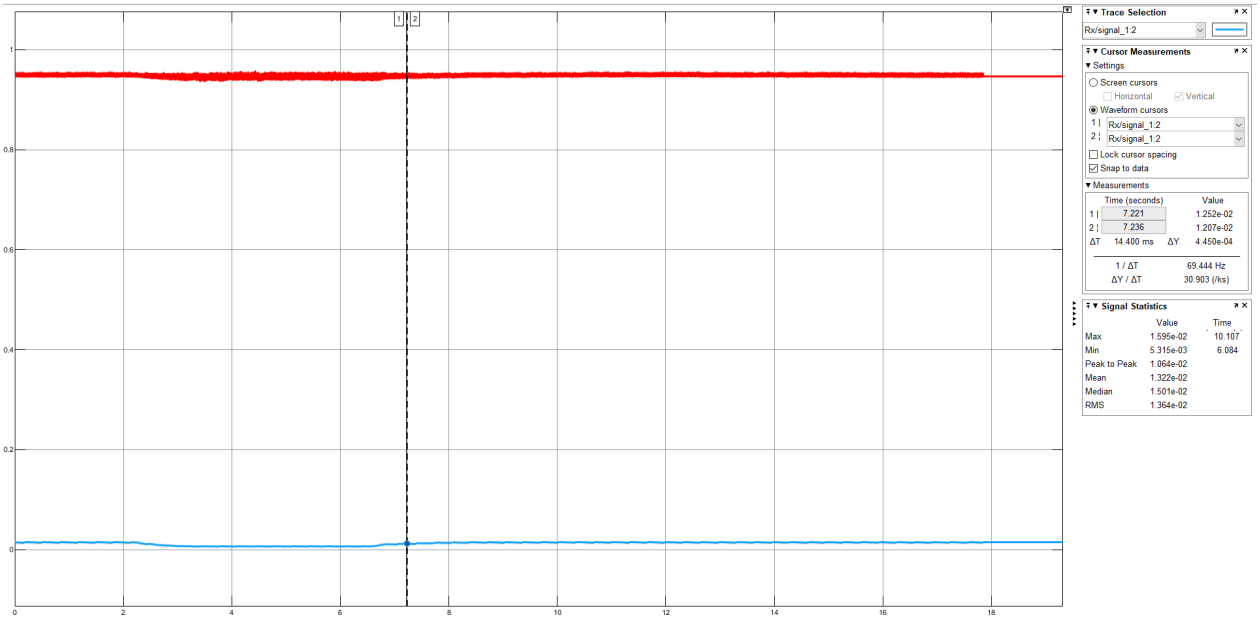

alpha1=alpha;
```

implementation on a matlab block of the theory

To have a noticeable difference among our L, R, parameters in a limited time period, we proceeded in the following way:

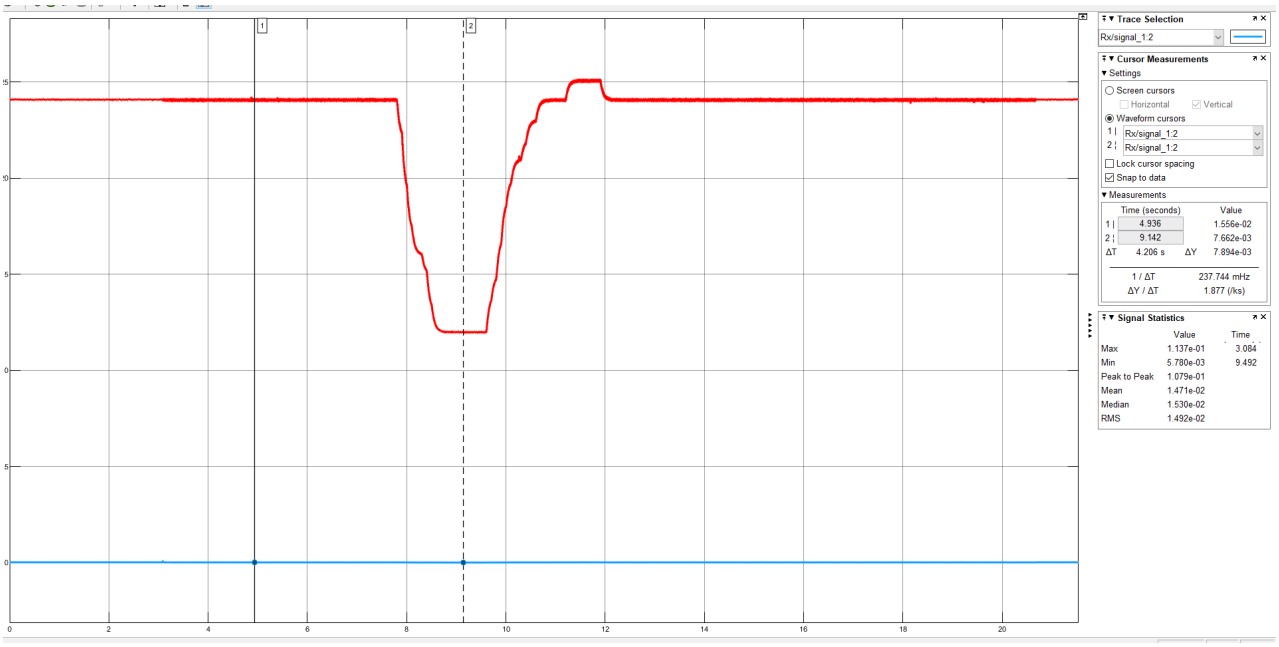
We fixed the expected input voltage of our system to 24V throughout the entirety of the experiment. At this point we “tricked” our system by varying manually the true input voltage from the power supply.

For sake of simplicity let’s follow with an example: by halving the input voltage manually, the input current also is halved, but since the system expects a total input voltage of 24V, it calculates that this variation is generated from a variation (doubling) of the Resistance/Inductance rather than a decrease in current due to the decrease of the voltage.

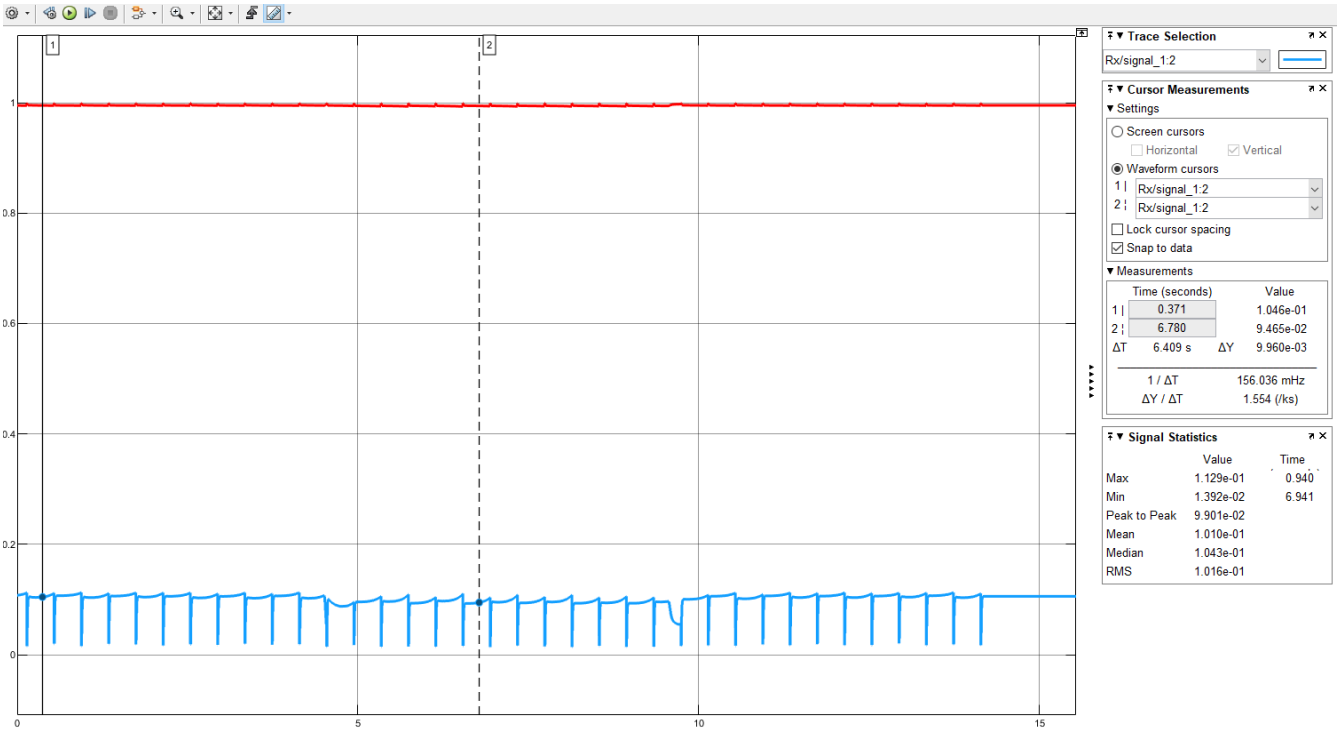


Evolution of α1(red) and α2 (blue) in time – Ther variation indicates the proper function of the estimation.

Samuele Pilati
Giorgio Giaretta
Tommaso Magelli



Evolution of α2(blue) with reference to Voltage (red) in time – From the measurements taken we can notice that when Input voltage is halved to 12V, also α2 gets halved.



Variation of α2 in J and b estimation with load connection (the blue line is α2 multiplied by k_m)

Samuele Pilati
Giorgio Giaretta
Tommaso Magelli

CONCLUSION

In summary, the laboratory work provided practical and well-rounded overview of the different techniques implemented in designing high-performance motion control systems. The implementation of a cascaded PI controller, tuned using experimentally identified parameters from the Least Squares method, clearly showed performance compared to controllers based on nominal datasheet values.

This baseline performance was significantly enhanced through advanced techniques. The addition of feedforward compensation proved critical for achieving high-precision trajectory tracking by tackling the fast-changing system's dynamics. Furthermore, the exploration of analytical controllers, specifically Deadbeat and Dahlin, highlighted into the design trade-offs between response speed and control effort.

Across all implementations, we demonstrated the necessity of anti-windup schemes by showcasing their effect in managing actuator saturation and ensuring stable and coherent control behavior.

Overall this course taught us the crucial and most widely used techniques in DC motor drives, stressing the importance and the challenges of working with real plants with respect to models and simulations only.